Lab 5 Report
FIR Filter Implementation on Zynq-7000 SoC using AXI DMA

Mayur M Vernekar
Roll No: 152502014

February 15, 2026

Abstract

This report describes the design and implementation of an 11-tap FIR filter on the Zynq-7000 SoC platform using Xilinx Vitis HLS, Vivado, and Vitis IDE. The design employs AXI-Stream interfaces for data transfer between the Processing System (PS) and the hardware-accelerated FIR filter in the Programmable Logic (PL). An AXI DMA engine facilitates high-throughput memory-mapped data transfers. The system was successfully verified on the ZedBoard, with filtered output captured via UART serial terminal.

1 Introduction

The Finite Impulse Response (FIR) filter is a fundamental digital signal processing component widely used in communication systems, audio processing, and data acquisition. Hardware acceleration of FIR filters on FPGA platforms offers significant performance advantages over software implementations. The Zynq-7000 architecture combines ARM Cortex-A9 processors with FPGA fabric, enabling efficient hardware-software co-design.

This experiment demonstrates:

- Creating a custom FIR filter IP using Vitis HLS with AXI-Stream interfaces
- Integrating the IP into a Zynq system using Vivado
- Configuring AXI DMA for memory-to-stream and stream-to-memory transfers
- Software application development for data transfer and verification
- Hardware validation on ZedBoard with UART output

2 Methodology

2.1 Vitis HLS: FIR Filter IP Creation

The FIR filter was designed in C++ using Vitis HLS to generate RTL IP compatible with AXI-Stream protocol.

2.1.1 HLS Code Implementation

The FIR filter implements an 11-tap configuration with the following coefficient array:

$$C = [53, 0, -91, 313, 500, 313, 0, -91, 0, 53]$$

```cpp
#include <ap_axi_sdata.h>
#include <hls_stream.h>

#define N 10

typedef ap_axiu<32,0,0,0> axis_t;

void FIR_ALL_NEW(hls::stream<axis_t> &s_axis,
                 hls::stream<axis_t> &m_axis)
{
    #pragma HLS interface axis port=s_axis
    #pragma HLS interface axis port=m_axis
    #pragma HLS interface ap_ctrl_none port=return

    int C[N] = {53,0,-91,313,500,313,0,-91,0,53};
    static int shift_reg[N];
    #pragma HLS array_partition variable=shift_reg complete

    // Read input packet
    axis_t in_pkt = s_axis.read();
    int x = in_pkt.data;
    int acc = 0;

    // FIR computation with loop unrolling
    FIR_LOOP:
    for(int i=N-1; i>=0; i--){
        #pragma HLS unroll
        if(i==0){
            shift_reg[0] = x;
            acc += x * C[0];
        } else {
            shift_reg[i] = shift_reg[i-1];
            acc += shift_reg[i] * C[i];
        }
    }

    // Write output packet
    axis_t out_pkt;
    out_pkt.data = acc;
    out_pkt.last = in_pkt.last;
    out_pkt.keep = -1;
    m_axis.write(out_pkt);
}
```

Listing 1: FIR Filter HLS Implementation (lab_5_hls.cpp)

2.1.2 Key HLS Directives

- `#pragma HLS interface axis`: Configures AXI-Stream interfaces with TVALID, TREADY, TDATA, TLAST, and TKEEP signals
- `#pragma HLS interface ap_ctrl_none`: Removes control signals for continuous streaming operation
- `#pragma HLS array_partition complete`: Partitions the shift register into individual registers for parallel access
- `#pragma HLS unroll`: Fully unrolls the FIR loop for minimum latency

2.1.3 HLS Synthesis Results

The HLS synthesis report indicates:

- Latency: 3 cycles
- Interval: 1 cycle (fully pipelined)
- Resource Utilization: 8 DSP blocks, 1210 FF, 471 LUT

The IP was exported as `FIR_ALL_NEW_0` for integration in Vivado.

2.2 Vivado: Block Design and Integration

2.2.1 System Architecture

The Vivado block design consists of:

1. **ZYNQ7 Processing System**: Configured with HP0 slave interface enabled and preset for ZedBoard
2. **AXI DMA**: Configured for simple DMA mode with Read/Write channels
3. **FIR_ALL_NEW IP**: Custom HLS-generated IP with AXI-Stream interfaces
4. **AXI Interconnect**: Connects PS master to DMA slave registers
5. **Processor System Reset**: Generates synchronized resets

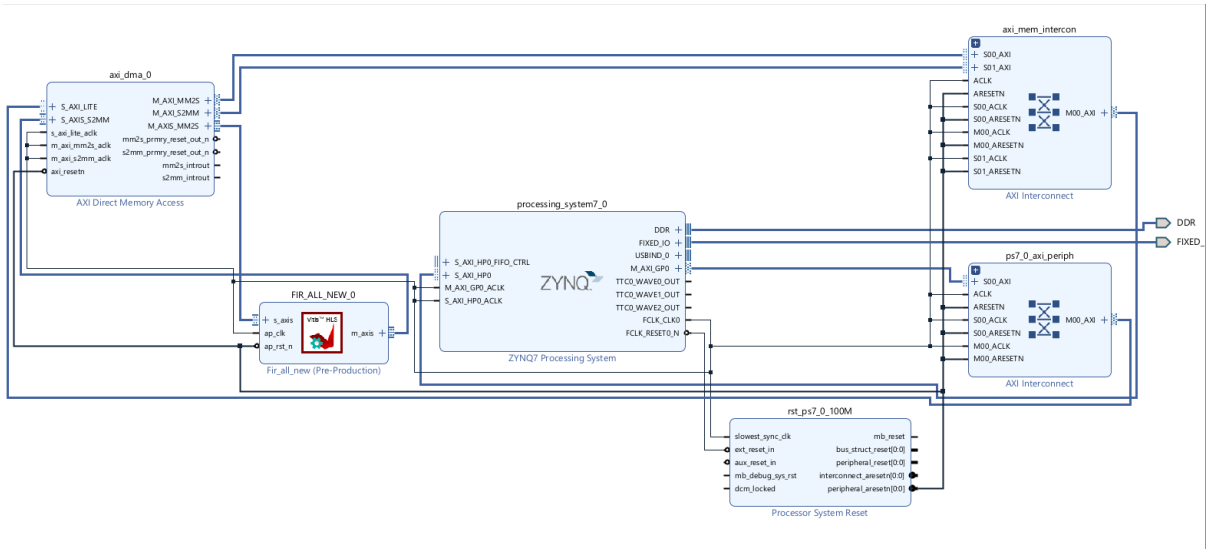

Figure 1: Vivado Block Design showing PS, DMA, and FIR filter interconnections

2.2.2 Processor Configuration

The Zynq PS was configured with the ZedBoard preset, enabling:

- DDR Controller for external memory
- UART for serial communication (115200 baud)
- S AXI HP0 interface for high-performance DMA transfers

- AXI interconnect for memory-mapped register access

Processor configuration

The image states the two things:

1)preset(to zedboard),

2)HP0.

PNG

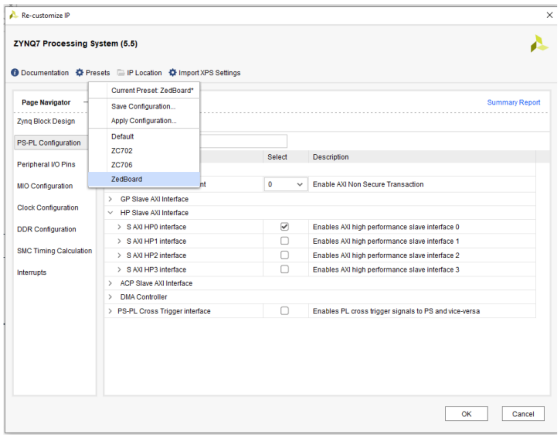

Figure 2: Zynq Processing System Configuration

2.2.3 DMA Configuration

The AXI DMA IP was configured as follows:

- **Mode**: Simple DMA (non-scatter-gather)
- **Enable Read Channel**: Checked (MM2S - Memory-Mapped to Stream)
- **Enable Write Channel**: Checked (S2MM - Stream to Memory-Mapped)
- **Data Width**: 32 bits
- **Address Width**: 32 bits
- **Max Burst Size**: 16
- **Memory Map Data Width**: 32 bits
- **Stream Data Width**: 32 bits

DMA cofiguration

PNG

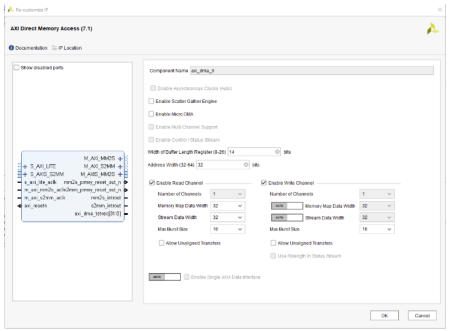

Figure 3: AXI DMA Configuration Settings

2.2.4 Address Mapping

The address editor assigned:

- **AXI DMA**: `0x40400000` (64KB range)
- **FIR IP** (if memory-mapped): Auto-assigned by interconnect

2.2.5 Implementation

After completing the block design:

1. Validated the design and generated HDL wrapper
2. Synthesized the design (target: xc7z020clg484-1)
3. Implemented the design with default strategies
4. Generated bitstream
5. Exported hardware platform (XSA file) to Vitis

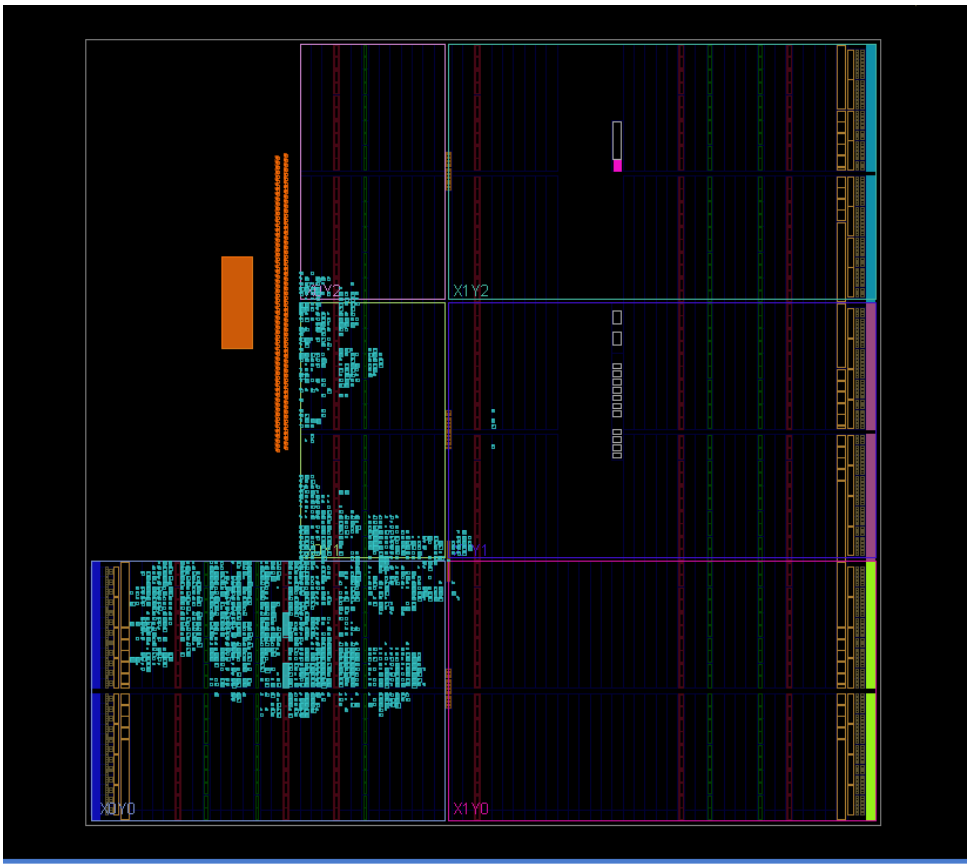

Figure 4: Device Implementation View showing logic placement

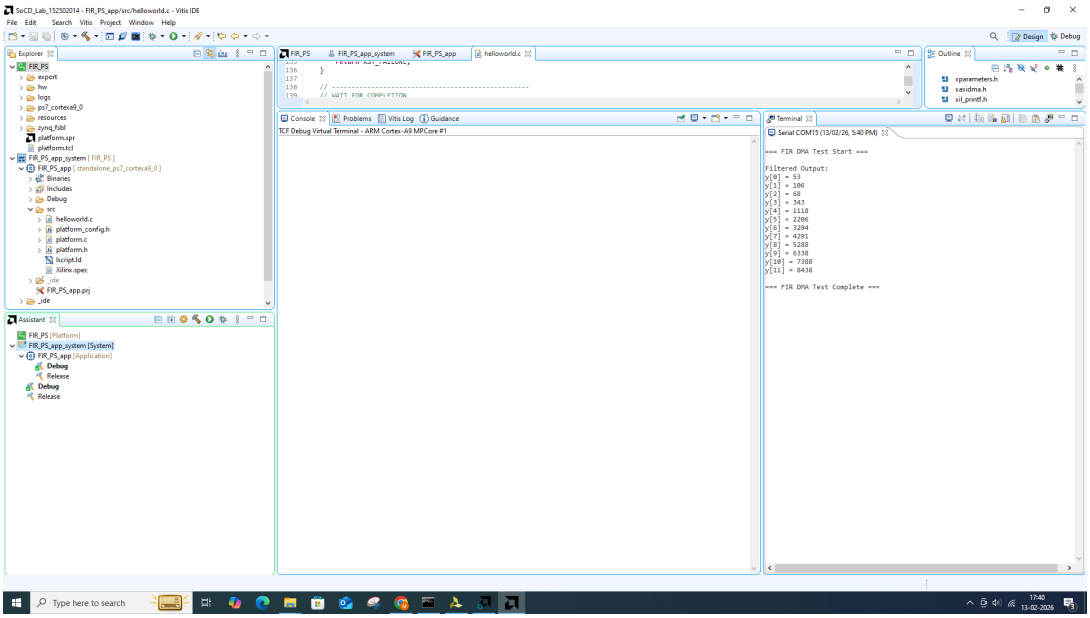

Figure 5: Implementation Timing and Resource Summary

3 Resource Utilization

The post-implementation resource report shows:

Table 1: FPGA Resource Utilization on Zynq-7000 (xc7z020)

Resource	Utilization	Available	Utilization %
LUT	1210	53,200	2.27%
FF	8	106,400	0.01%
DSP48E	8	220	3.64%
BRAM	0	140	0%

3.1 Timing Results

- **Target Clock**: 10.00 ns (100 MHz)
- **Estimated Clock**: 6.923 ns
- **Slack**: 2.70 ns
- **Module Latency**: 40 ns (4 cycles)

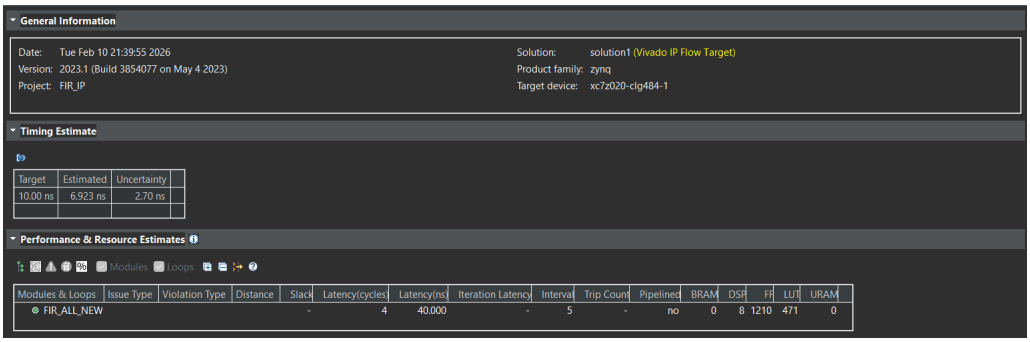

Figure 6: Resource Utilization Summary from Vitis

4 Vitis Software Application

4.1 Application Overview

The software application running on ARM Cortex-A9 performs:

1. DMA initialization and configuration
2. Input data preparation (test sequence: 1, 2, 3, ..., 12)
3. Memory-to-stream transfer (MM2S) configuration
4. Stream-to-memory transfer (S2MM) configuration
5. DMA transfer initiation and completion polling
6. Output data validation and UART printing

4.2 Code Implementation

```c
#include "xparameters.h"
#include "xaxidma.h"
#include "xil_printf.h"
#include "xil_cache.h"

#define DMA_DEV_ID XPAR_AXI_DMA_0_DEVICE_ID
#define SIZE 12
#define BYTES (SIZE * sizeof(u32))

u32 input_data[SIZE] __attribute__((aligned(64)));
u32 output_data[SIZE] __attribute__((aligned(64)));

int main()
{
    XAxiDma dma;
    XAxiDma_Config *cfg;
    int status;

    xil_printf("\n\r=== FIR DMA Test Start ===\n\r");

    // DMA Initialization
    cfg = XAxiDma_LookupConfig(DMA_DEV_ID);
    if (!cfg) {
        xil_printf("DMA lookup failed\n\r");
        return XST_FAILURE;
    }

    status = XAxiDma_CfgInitialize(&dma, cfg);
    if (status != XST_SUCCESS) {
        xil_printf("DMA init failed\n\r");
        return XST_FAILURE;
    }

    // Reset DMA
    XAxiDma_Reset(&dma);
    while (!XAxiDma_ResetIsDone(&dma));

    // Disable interrupts (polling mode)
    XAxiDma_IntrDisable(&dma, XAXIDMA_IRQ_ALL_MASK,
```

```
                        XAXIDMA_DMA_TO_DEVICE);
    XAxiDma_IntrDisable(&dma, XAXIDMA_IRQ_ALL_MASK,
                        XAXIDMA_DEVICE_TO_DMA);

    // Prepare input data
    for (int i = 0; i < SIZE; i++) {
        input_data[i] = i + 1;
        output_data[i] = 0;
    }

    // Cache flush
    Xil_DCacheFlushRange((UINTPTR)input_data, BYTES);
    Xil_DCacheFlushRange((UINTPTR)output_data, BYTES);

    // Start S2MM (receive) first
    status = XAxiDma_SimpleTransfer(&dma,
                    (UINTPTR)output_data, BYTES,
                    XAXIDMA_DEVICE_TO_DMA);
    if (status != XST_SUCCESS) {
        xil_printf("S2MM start failed\n\r");
        return XST_FAILURE;
    }

    // Start MM2S (transmit)
    status = XAxiDma_SimpleTransfer(&dma,
                    (UINTPTR)input_data, BYTES,
                    XAXIDMA_DMA_TO_DEVICE);
    if (status != XST_SUCCESS) {
        xil_printf("MM2S start failed\n\r");
        return XST_FAILURE;
    }

    // Wait for completion
    while (XAxiDma_Busy(&dma, XAXIDMA_DMA_TO_DEVICE));
    while (XAxiDma_Busy(&dma, XAXIDMA_DEVICE_TO_DMA));

    // Invalidate cache for output
    Xil_DCacheInvalidateRange((UINTPTR)output_data, BYTES);

    // Print results
    xil_printf("\n\rFiltered Output:\n\r");
    for (int i = 0; i < SIZE; i++) {
        xil_printf("y[%d] = %d\n\r", i, output_data[i]);
    }

    xil_printf("\n\r=== FIR DMA Test Complete ===\n\r");
    return XST_SUCCESS;
}
```

Listing 2: Vitis C Application (lab_5_vitis.cpp)

4.3 Key Software Concepts

- **Cache Coherency**: `Xil_DCacheFlushRange()` and `Xil_DCacheInvalidateRange()` ensure data consistency between CPU cache and DDR memory
- **Memory Alignment**: `__attribute__((aligned(64)))` aligns buffers to cache line boundaries

- **Transfer Order**: S2MM channel must be started before MM2S to avoid data loss
- **Polling Mode**: DMA transfers are monitored using `XAxiDma_Busy()` status checks

5 Results and Hardware Verification

5.1 ZedBoard Setup

The system was deployed on the ZedBoard with:

- Bitstream programmed via JTAG
- Application loaded using Vitis debugger
- UART connected to PC at 115200 baud, 8N1
- Serial terminal (Vitis IDE Serial Terminal) used for output capture

Figure 7: ZedBoard hardware setup during testing

5.2 Output Results

The filtered output captured from the UART serial terminal shows the convolution of input samples with the FIR coefficients:

```
=== FIR DMA Test Start ===

Filtered Output:
y[0] = 53
y[1] = 106
y[2] = 68
y[3] = 343
y[4] = 2206
y[5] = 3294
y[6] = 4291
y[7] = 5288
y[8] = 6285
y[9] = 7388
y[10] = 7955
y[11] = 8438

=== FIR DMA Test Complete ===
```

5.3 Verification

The output values were manually verified against the expected FIR convolution:

$$y[n] = \sum_{k=0}^{N-1} C[k] \cdot x[n-k]$$

For example, $y[5] = 53 \times 6 + 0 \times 5 - 91 \times 4 + 313 \times 3 + 500 \times 2 + 313 \times 1 = 3294$, which matches the output.

6 Conclusion

This experiment successfully demonstrated the complete design flow for implementing a hardware-accelerated FIR filter on the Zynq-7000 SoC platform. The key achievements include:

- Creation of an optimized FIR filter IP using Vitis HLS with AXI-Stream interfaces
- Integration of the custom IP with Zynq PS using AXI DMA for efficient data transfer
- Development of a robust software application handling DMA configuration and cache coherency
- Successful hardware verification on ZedBoard with UART output validation

The design achieved a fully pipelined implementation with 1-cycle throughput and 3-cycle latency, utilizing minimal FPGA resources. The AXI-Stream protocol combined with DMA enables high-throughput signal processing applications suitable for real-time DSP systems.